SCIM250

AY25/26 SEM 1

nadear

Lists

```
# 1. Load & Quick Explore
df = pd.read_csv("data.csv") # Load CSV
df.shape # (rows, cols)
df.columns # column names
df.info() # dtypes + missing
df.describe(include="all") # stats (num + cat)
df.head() ; df.tail() # first 5rows ; last 5rows
df.isna().sum() # missing count per column
df['City'].value_counts() # freq table of a column
df.corr(numeric_only=True) # correlation (numeric cols)
# 2. Missing Values
df = df.dropna() # drop any row with NaN in ANY column
# Drop rows where specific columns have NaN (Age, Salary)
df = df.dropna(subset=['Age', 'Salary']) # temporary
df.dropna(subset=['Age', 'Salary'], inplace=True) # permanent
# Impute using mean / median / mode
df['Age'] = df['Age'].fillna(df['Age'].mean()) # mean
df['Score'] = df['Score'].fillna(df['Score'].median())# median
df['City'] = df['City'].fillna(df['City'].mode()[0]) # mode
# 3. Convert Types
df['Price'] = pd.to_numeric(df['Price'],errors='coerce') # numeric
df['Date'] = pd.to_datetime(df['Date'],format='mixed') # datetime
# Rename & select columns
df = df.rename(columns={'old_name': 'new_name'}) # rename col
df = df[['Name', 'Age', 'Score', 'City', 'Price', 'Date']]
# 4. Handle Duplicates
df = df.drop_duplicates() # all columns
df = df.drop_duplicates(subset=['Name', 'Date']) # specific cols
# 5. Outliers (IQR rule)
Q1 = df['Score'].quantile(0.25)
Q3 = df['Score'].quantile(0.75)
IQR = Q3 - Q1
df = df[(df['Score'] >= Q1 - 1.5 * IQR) &
        (df['Score'] <= Q3 + 1.5 * IQR)]
# 6. Filtering / Conditions
df = df[df['Age'] > 18] # keep only Age > 18
df = df[(df['Salary'] >= 20000) & (df['Salary'] <= 80000)]
# 7. New Columns / Feature Eng.
df['Age_Group'] = pd.cut(df['Age'], bins=[0, 18, 30, 50, 100],
       labels=['child', 'young', 'adult', 'senior'])
df['Income_per_Year'] = df['Salary'] * 12
# Map / replace values
df['City_Short'] = df['City'].map({'Bangkok': 'BKK', 'Chiang Mai':
        'CNX'}).fillna('OTHER')
# 8. Groupby & Aggregation
city_stats = df.groupby('City')['Score'].agg(['mean', 'count', '
       min', 'max'])
age_stats = df.groupby('Age_Group')['Salary'].mean()
# Final check
df.info() # overall
df.describe() # statistic
```

Visualization

```
# 1. Histogram
plt.figure()
plt.hist(df["col"], bins=30, edgecolor="black")
plt.xlabel("x"); plt.ylabel("y"); plt.title("Histogram")
plt.show()
sns.histplot(df["col"], bins=30, kde=True, color="lightgreen",
       edgecolor="red")
# 2. Line plot
plt.plot(x, y, marker="o", linestyle="--", linewidth=2)
# 3. Scatter plot
plt.scatter(x, y)
sns.scatterplot(data=df, x="col_x", y="col_y", hue="category")
# 4. Pie chart
plt.pie([1, 2, 7], labels=["coffee", "green tea", "chocolate"],
       autopct="%1.1f%%")
# 5. Bar chart
plt.bar([1, 2, 3], [10, 15, 20])
# 6. Horizontal bar chart
plt.barh(range(4), [9, 7, 2, 3], tick_label=["a", "b", "c", "d"])
# 7. Box plot
plt.boxplot([large_dist, small_dist])
plt.xticks([1, 2], ["large", "small"])
# 8. Violin plot
plt.violinplot([large_dist, small_dist])
# 9. Pairplot (scatter matrix)
sns.pairplot(data=df)
# 10. Correlation + heatmap
data_corr = df.corr(numeric_only=True)
sns.heatmap(data_corr, vmin=-1, vmax=1,
            annot=True, cmap="coolwarm", square=True)
# Style cheats:
# marker="o" dot at each point
# linestyle="--" dashed line
# color="r" red
# linewidth=2 thicker line
# alpha=0.5 transparency
```

Standard Scaler

A feature scaling technique which follows Standard Normal Distribution and is used to standardize the values of numeric features. It transforms data so that the mean becomes 0 and the standard deviation becomes 1.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train_scaled = scaler.fit_transform(X_train) # fit on train
X_val_scaled = scaler.transform(X_val) # transform val
X_test_scaled = scaler.transform(X_test) # transform test
```

Statsmodels

```
import statsmodels.api as sm
from statsmodels.tsa.seasonal import seasonal_decompose
decomposition = seasonal_decompose(df["interpolated"], model="
       additive", extrapolate_trend="freq")
# This produces four plots: observed, trend, seasonal, residual.
       Decomposition component:
observed = decomposition.observed
trend = decomposition.trend
seasonal = decomposition.seasonal
residuals = decomposition.resid
# Visualize the observed and trend components on the same graph.
df_decomposed = pd.DataFrame(
    np.c_[observed, trend], # combine arrays column-wise
    index=df2.index, # keep the same datetime index
    columns=['observed','trend']) # label the two columns
```

Supervised Learning

In supervised learning, the training set you feed to the algorithm includes the desired solutions, called labels.

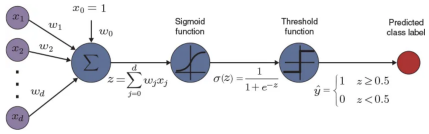
Linear Regression

Regression algorithm to predict continuous output values by fitting a linear equation that minimizes the sum of squared residuals.

```
from sklearn.linear_model import LinearRegression
X = np.c_[x1, x2]
model = LinearRegression()
model.fit(X, y)
```

Logistic Regression

a type of classification algorithm that predicts a discrete or categorical outcome.




```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(max_iter=10000, random_state=0)
clf.fit(X_train, y_train)
```

Lasso Regression

A regularization technique that applies a penalty to prevent overfitting and enhance the accuracy of statistical models.

```
from sklearn.linear_model import Ridge, Lasso
model = Lasso(alpha=0.1)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Ridge Regression

A statistical regularization technique. It corrects for overfitting on training data in machine learning models.

```
from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Non-parametric Supervised Learning

Decision Trees

Used for both regression and classification; model decisions and outcomes using a tree-like structure of splits. Components of Decision Trees:

- **Leaves** — represent the predicted class labels or output values.
- **Nodes** — contain attribute-based test, with branches for possible outcomes.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
Tree = DecisionTreeClassifier(criterion="entropy", max_depth = 4)
Tree.fit(X_train,y_train)
tree_predictions = Tree.predict(X_test)
# Visualize the tree
plot_tree(Tree)
plt.show()
```

K-Nearest Neighbors (KNN)

Non-parametric method for both tasks; predicts a class or value based on the majority vote or average of the k closest neighbors.

Parameters

- `n_neighbors` - Number of nearest neighbors to use for predictions.
- `weights` - Weight function to be applied to the nearest neighbors.
- `algorithm` - The algorithm used to compute the nearest neighbors.
- `metric` - The metric used for distance computations.

```
from sklearn.neighbors import KNeighborsClassifier
k = 3
#Train Model and Predict
knn_classifier = KNeighborsClassifier(n_neighbors=k)
knn_model = knn_classifier.fit(X_train,y_train)

yhat = knn_model.predict(X_test)
```

Unsupervised Learning

In unsupervised learning, the training data is unlabeled. The system tries to learn without a teacher.

K-means - Clustering

Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

```
from sklearn.cluster import KMeans
# Define the number of clusters (10 for digits 0-9)
n_clusters = 10
# Initialize and fit KMeans
kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init='
       auto', max_iter=10)
kmeans_labels = kmeans.fit_predict(X)

labels = kmeans.labels_
centers = kmeans.cluster_centers_
```

DBSCAN

a density-based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space.

```
from sklearn.cluster import DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X_scaled)
```

PCA - Dimensionality Reduction

A dimensionality reduction technique and helps us to reduce the number of features in a dataset while keeping the most important information. It changes complex datasets by transforming correlated features into a smaller set of uncorrelated components.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

components = pca.components_
pca.explained_variance_ratio_
```

t-SNE

```
from sklearn.manifold import TSNE
n_components = 2
learning_rate = 300
perplexity = 30
early_exaggeration = 12
init = 'random'
random_state = 42
tSNE = TSNE(n_components=n_components, learning_rate=learning_rate
       , perplexity=perplexity, early_exaggeration=
       early_exaggeration, init=init, random_state=random_state)

df_tSNE = tSNE.fit_transform(df)
```

Binning or Discretization

```
from sklearn.preprocessing import KBinsDiscretizer
preprocessor = ColumnTransformer([('discretizer', KBinsDiscretizer
       (n_bins=10), columns),], remainder='passthrough')
```

Semi-supervised Learning

Since labeling data is usually time-consuming and costly, you will often have plenty of unlabeled instances and few labeled instances. Some algorithms can deal with data that's partially labeled.

Self-supervised learning

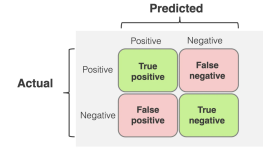
Self-supervised learning is a trick where the model makes its own labels from raw, unlabeled data.

Reinforcement learning

Is a type of machine learning process in which autonomous agents learn to make decisions by interacting with their environment.

Evaluation Metrics

Confusion Matrix




```
from sklearn.metrics import confusion_matrix,
       ConfusionMatrixDisplay
y_test_pred = model.predict(X_test)
conf_mat = confusion_matrix(y_test, y_test_pred)
print(conf_mat)
# A visualized confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=conf_mat)
disp.plot(cmap='Blues')
```

Mean Square Error (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Here,

- n is the number of data points,
- y_i is the actual (observed) value for the i-th data point,
- $\hat{y}_i$ is the predicted value for the i-th data point.

MSE is a way to quantify the accuracy of a model's predictions. MSE is sensitive to outliers as large errors contribute significantly to the overall score.

```
from sklearn.metrics import mean_squared_error
# Given values
Y_true = [1,1,2,2,4] # Y_true = Y (original values)
# calculated values
Y_pred = [0.6,1.29,1.99,2.69,3.4] # Y_pred = Y'
# Calculation of Mean Squared Error (MSE)
mean_squared_error(Y_true,Y_pred)
```

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|Y_i - \hat{Y}_i\right|$$

Here,

- n is the number of observations,
- Y_i represents the actual values,
- $\hat{Y}_i$ represents the predicted values.

A lower MAE indicates better model performance. Unlike MSE, MAE is less sensitive to outliers since it uses absolute differences.

```
from sklearn.metrics import mean_absolute_error as mae
# list of integers of actual and calculated
actual = [2, 3, 5, 5, 9]
calculated = [3, 3, 8, 7, 6]
# calculate MAE
error = mae(actual, calculated)
```

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{\text{RSS}}{n}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(y_i^{\text{actual}} - y_i^{\text{predicted}}\right)^2}$$

Where
- n : number of observations,
- y_i^{actual} : actual value of the i-th data point,
- $y_i^{\text{predicted}}$: predicted value of the i-th data point.

RMSE has the same units as the target variable and emphasizes larger errors more than smaller ones.

```
from sklearn.metrics import root_mean_squared_error as RMSE
y_pred = model.predict(X)
rmse = RMSE(y, y_pred)
print(f'RMSE: {rmse:.4f}')
```

Coefficient of Determination (R-squared)

R-squared is a statistic that indicates how much of the variation in the dependent variable is explained by the model. Its value lies between 0 and 1; higher values generally mean a better fit.

$$R^2 = 1 - \left(\frac{\text{RSS}}{\text{TSS}}\right) = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where RSS is the residual sum of squares and TSS is the total sum of squares.

```
r2_score = model.score(X, y)
print(f'R2 score: {r2_score:.4f}')
```

Accuracy

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

```
from sklearn.metrics import accuracy_score
acc = accuracy_score(y_test, clf.predict(X_test)) * 100
```

Precision

$$\text{precision} = \frac{TP}{TP + FP}$$

```
from sklearn.metrics import precision_score
precision = precision_score(y_test, y_test_pred)
```

Recall

$$\text{recall} = \frac{TP}{TP + FN}$$

```
from sklearn.metrics import recall_score
recall = recall_score(y_test, y_test_pred)
```

F-Scores

F1 score is the harmonic mean of Precision and Recall; it punishes models that do well on only one of them. Both Precision and Recall must be high for the F1 score to be high.

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

```
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_test_pred)
```

ROC Curve

used to check how well a binary classification model works. **X-axis:** False Positive Rate

$$\text{FPR} = \frac{FP}{FP + TN}$$

Y-axis: True Positive Rate (Recall)

$$\text{TPR} = \frac{TP}{TP + FN}$$

Each point on the curve is a (FPR, TPR) pair for a particular decision threshold.

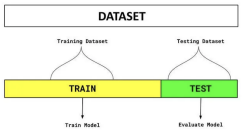
```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_test, probs)

plt.plot(fpr, tpr, linewidth=2)
plt.plot([0, 1], [0, 1], 'k--')
plt.axis([0, 1, 0, 1])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
```

Area Under the ROC curve (AUC)

```
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y_test, probs)
```

Validation methods in Machine Learning

Hold-out validation



Hold-out Validation the dataset is typically divided into three parts:

1. **Training set (typically 60–80% of data)**: Used to train the model by adjusting its parameters.
2. **Validation set (typically 10–20% of data)**: Used to tune hyperparameters and make decisions about model architecture.
3. **Test set (typically 10–20% of data)**: Used only once at the end to evaluate the final model's performance.

```
from sklearn.model_selection import train_test_split
X = df.drop(columns=['target'])
y = df['target']
# First split: train+val vs test (80% / 20%)
X_train, X_test, y_train, y_test = train_test_split(X, y,
       test_size=0.4, random_state=0)
# Second split: train vs validation (from the 80%) (if needed)
# 0.25 of 0.8 = 0.2 ; final: 60% train, 20% val, 20% test
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train
       , test_size=0.25, random_state=42, stratify=y_train_full)
```

Bootstrapping Validation

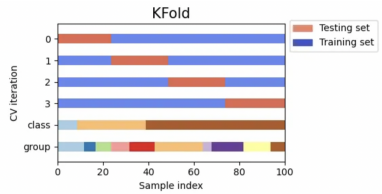
A resampling technique that creates multiple datasets by randomly sampling from the original dataset with replacement.


```
from sklearn.utils import resample
bootstrap_idx = resample(range(len(X)), replace=True, n_samples=
       len(X), random_state=42)
oob_idx = list(set(range(len(X))) - set(bootstrap_idx))

X_train = X.iloc[bootstrap_idx]
y_train = y.iloc[bootstrap_idx]
X_oob = X.iloc[oob_idx]
y_oob = y.iloc[oob_idx]
```

Cross Validation

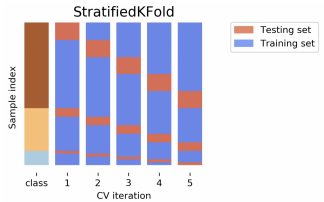
A technique used to evaluate machine learning models and ensure they generalize well to unseen data.

K Fold Cross Validation The standard approach where data is divided into k equal folds (typically k=5 or k=10)



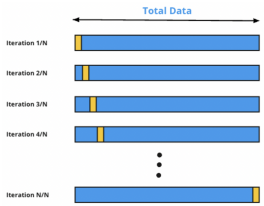

```
from sklearn.model_selection import KFold
kf = KFold(n_splits=5, shuffle=True, random_state=42)

fold = 1
for train_idx, test_idx in kf.split(df):
    train_df = df.iloc[train_idx]
    test_df = df.iloc[test_idx]
    fold += 1
```

Stratified K-Fold Similar to k-fold, but ensures each fold has approximately the same percentage of samples of each target class (important for imbalanced datasets)




```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
fold = 1
for train_idx, test_idx in skf.split(X, y):
    train_df = df.iloc[train_idx]
    test_df = df.iloc[test_idx]
    fold += 1
```

Leave-One-Out (LOO) An extreme case where k equals the number of samples, so each training set contains all but one sample

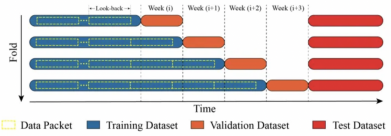



```
from sklearn.model_selection import LeaveOneOut
loo = LeaveOneOut()
for i, (train_idx, test_idx) in enumerate(loo.split(X)):
    print(f"Iteration {i+1}")
    print("Train indices:", train_idx)
    print("Test index:", test_idx)
    print()
```

Time Series Cross-Validation Method

Expanding Window approach

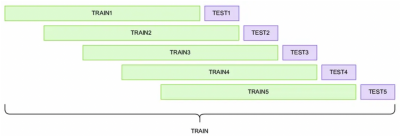
A time-series evaluation method where the training set grows over time: at each step the model is trained on all data up to time t and tested on the next period. It respects temporal order (no future data used for training), mimics how models are updated in reality, and is useful when patterns are fairly stable and you want to exploit all available historical information.




```
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=4, test_size=2)
# 4folds, each test 2samples
fold = 1
for train_idx, test_idx in tscv.split(data):
    train = data.iloc[train_idx]
    test = data.iloc[test_idx]
    fold += 1
```

Rolling Window approach

The training and test datasets are kept at a fixed size. At each step, this window is shifted forward, so the same amount of training data is used in each step.




```
window_size = 6# training window size
test_size = 2
step_size = test_size
n_samples = len(data)

fold = 1
for start in range(0, n_samples - window_size - test_size + 1,
       step_size):
    train = data.iloc[start:start + window_size]
    test = data.iloc[start + window_size:start + window_size +
          test_size]
    fold += 1
```

Pipeline

a module in scikit-learn that lets you chain multiple steps (preprocessing + model) into one object, called a Pipeline.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('scaler', StandardScaler()), # step 1: scale features
    ('logreg', LogisticRegression())]) # step 2: classifier
# Fit everything in one go
pipe.fit(X_train, y_train)
# Automatically scales X_test then predicts
y_pred = pipe.predict(X_test)
```

SQLite

A simple, file-based SQL database built into Python. No server, no setup — just a single portable .db file.

Pros(Strengths)
- Easy to use, no configuration.
- Fast for reading data.
- Very portable (one file).
- Lightweight & ACID-compliant.

Cons(Limitations)
- Poor for high concurrency.
- Not for large-scale/multi-user.
- Fewer advanced features.
- No built-in user management.

Initiate the connector

```
# Adjust these values for your PostgreSQL setup
host = 'localhost'
port = '5432'
dbname = 'analysis'
user = 'postgres'
password = 'password'

pg_conn = psycopg2.connect(host=host, port=port, dbname=dbname,
       user=user, password=password)
```

Create a table

```
create_table_sql = '''
CREATE TABLE IF NOT EXISTS movie(
        title TEXT,
        year INTEGER,
        score DOUBLE PRECISION);
'''
pg_conn.commit()
pg_cursor.close()
```

Insert data

```
insert_pg_sql = '''
INSERT INTO movie (title, "year", score) VALUES
    ('Monty Python and the Holy Grail', 1975, 8.2),
    ('And Now for Something Completely Different', 1971, 7.5);
'''
pg_cursor = pg_conn.cursor()
pg_cursor.execute(insert_pg_sql)
pg_conn.commit()
pg_cursor.close()
```